

Timesheet, Expense Claims & Leave Management Application (Project Outline, File Structure & Team Tasks)

Generated: October 21, 2025

Tech Stack

- Backend: NestJS + Drizzle ORM + PostgreSQL
- Frontend: React.js + Vite + Tailwind CSS
- Auth: JWT + Google SSO
- Storage: AWS S3
- Deployment: AWS (EC2 / Amplify)
- Version Control: GitHub (branch per module)

Recommended Backend File Structure (NestJS + Drizzle + PostgreSQL)

```
backend/  
  src/  
    main.ts  
    app.module.ts  
    config/  
      database.config.ts  
      s3.config.ts  
      auth.config.ts  
    common/  
      guards/  
      interceptors/  
      decorators/  
      utils/  
    auth/  
      auth.module.ts  
      auth.controller.ts  
      auth.service.ts  
      jwt.strategy.ts  
      google.strategy.ts  
      dto/  
      users/  
        users.module.ts  
        users.controller.ts  
        users.service.ts  
      entities/  
      dto/  
    timesheet/  
      timesheet.module.ts  
      timesheet.controller.ts  
      timesheet.service.ts  
      entities/  
      dto/  
    expenses/  
      expenses.module.ts  
      expenses.controller.ts  
      expenses.service.ts  
      entities/  
      dto/  
    leaves/  
      leaves.module.ts
```

- ■ ■ ■ ■ leaves.controller.ts
- ■ ■ ■ ■ leaves.service.ts
- ■ ■ ■ ■ entities/
- ■ ■ ■ ■ dto/
- ■ ■ ■ ■ reports/
- ■ ■ ■ ■ reports.module.ts
- ■ ■ ■ ■ reports.controller.ts
- ■ ■ ■ ■ reports.service.ts
- ■ ■ ■ ■ excel-export.util.ts
- ■ ■ ■ ■ drizzle/
- ■ ■ ■ ■ schema.ts
- ■ ■ ■ ■ migrations/
- ■ ■ ■ ■ drizzle.config.ts
- ■ ■ ■ ■ .env
- ■ ■ ■ ■ package.json
- ■ ■ ■ ■ tsconfig.json

Recommended Frontend File Structure (React + Tailwind)

- frontend/
- ■ ■ ■ ■ src/
- ■ ■ ■ ■ main.jsx
- ■ ■ ■ ■ App.jsx
- ■ ■ ■ ■ assets/
- ■ ■ ■ ■ components/
- ■ ■ ■ ■ Navbar.jsx
- ■ ■ ■ ■ Sidebar.jsx
- ■ ■ ■ ■ ProtectedRoute.jsx
- ■ ■ ■ ■ Table.jsx
- ■ ■ ■ ■ Form.jsx
- ■ ■ ■ ■ context/
- ■ ■ ■ ■ AuthContext.jsx
- ■ ■ ■ ■ APIProvider.jsx
- ■ ■ ■ ■ pages/
- ■ ■ ■ ■ Login.jsx
- ■ ■ ■ ■ Dashboard.jsx
- ■ ■ ■ ■ Timesheet/
- ■ ■ ■ ■ TimesheetList.jsx
- ■ ■ ■ ■ TimesheetForm.jsx
- ■ ■ ■ ■ TimesheetApproval.jsx
- ■ ■ ■ ■ Expenses/
- ■ ■ ■ ■ ExpenseList.jsx
- ■ ■ ■ ■ ExpenseForm.jsx
- ■ ■ ■ ■ ExpenseApproval.jsx
- ■ ■ ■ ■ Leaves/
- ■ ■ ■ ■ LeaveList.jsx
- ■ ■ ■ ■ LeaveForm.jsx
- ■ ■ ■ ■ LeaveApproval.jsx
- ■ ■ ■ ■ Users/
- ■ ■ ■ ■ UserList.jsx
- ■ ■ ■ ■ UserForm.jsx
- ■ ■ ■ ■ Reports/
- ■ ■ ■ ■ ExportReports.jsx
- ■ ■ ■ ■ services/

- ■ ■ ■ ■ api.js
- ■ ■ ■ ■ timesheetService.js
- ■ ■ ■ ■ expenseService.js
- ■ ■ ■ ■ leaveService.js
- ■ ■ ■ ■ userService.js
- ■ ■ ■ ■ reportService.js
- ■ ■ ■ ■ hooks/
- ■ ■ ■ ■ useFetch.js
- ■ ■ ■ ■ utils/
- ■ ■ ■ ■ constants.js
- ■ ■ ■ ■ styles/
- ■ ■ ■ ■ index.css
- ■ ■ ■ ■ .env
- ■ ■ ■ ■ vite.config.js
- ■ ■ ■ ■ package.json

Module Breakdown

Auth Module: JWT + Google SSO login
 User Management: Manage users & hierarchy
 Timesheet Module: Work hour tracking + approvals
 Expense Module: Expense claim management + proof upload
 Leave Module: Leave requests & tracking + balances
 Reports Module: Admin exports (Excel)

Team Division (4 Members)

Member 1 – Auth & User Management:
 - Backend: Auth module, JWT & Google SSO, user CRUD
 - Frontend: Login page, user management pages

Member 2 – Timesheet Module:
 - Backend: Timesheet entities, services, approval APIs
 - Frontend: Timesheet forms, lists, manager approval UI

Member 3 – Expense Module + AWS S3:
 - Backend: Expense APIs, file upload to S3, approval logic
 - Frontend: Expense form, proof upload, approval UI

Member 4 – Leave Module + Reports:
 - Backend: Leave CRUD, approval, Excel export service
 - Frontend: Leave form/list/approval, export UI (Admin)

Shared tasks: common setup, API integration, testing, deployment, docs

Suggested Drizzle Schema (snippet)

```
``ts
import { pgTable, varchar, integer, date, boolean, serial, text, numeric, timestamp } from "drizzle-orm/pg-core";

export const users = pgTable("users", {
  id: serial("id").primaryKey(),
  name: varchar("name").notNull(),
  email: varchar("email").unique().notNull(),
  role: varchar("role").default("Employee"),
  managerId: integer("manager_id").references(() => users.id),
```

```

});

export const timesheets = pgTable("timesheets", {
  id: serial("id").primaryKey(),
  userId: integer("user_id").references(() => users.id),
  project: varchar("project"),
  date: date("date").notNull(),
  hours: integer("hours").notNull(),
  description: text("description"),
  status: varchar("status").default("Pending"),
});

export const expenses = pgTable("expenses", {
  id: serial("id").primaryKey(),
  userId: integer("user_id").references(() => users.id),
  date: date("date").notNull(),
  category: varchar("category"),
  amount: numeric("amount", 10, 2),
  proofUrl: varchar("proof_url"),
  status: varchar("status").default("Pending"),
});

export const leaves = pgTable("leaves", {
  id: serial("id").primaryKey(),
  userId: integer("user_id").references(() => users.id),
  type: varchar("type"),
  startDate: date("start_date"),
  endDate: date("end_date"),
  reason: text("reason"),
  status: varchar("status").default("Pending"),
});

export const auditLogs = pgTable("audit_logs", {
  id: serial("id").primaryKey(),
  action: text("action"),
  userId: integer("user_id"),
  timestamp: timestamp("timestamp").defaultNow(),
});

```

Next Steps / Suggestions

- I can also generate: API endpoint list, Postman collection, GitHub branching strategy, or a dev timeline (sprints)
- Tell me which of those you'd like next and I will include it in the PDF if required.